
NanoCloud01: Data-Aware Continuation Training for a 99M-Parameter Language Model

Yirui Miao Fengfei Yu Tianyue Zhang Xavier Zhao

University of California San Diego

y5miao@ucsd.edu, fey006@ucsd.edu, tiz052@ucsd.edu, zhz214@ucsd.edu

<https://github.com/tylerz/cse251b-nanogpt>

Abstract

We study autoregressive language modeling under a strict 100M-parameter submission limit. Starting from a nanoGPT-style baseline, we investigate three coupled design axes: capacity allocation in compact Transformers, mixed-domain training data, and optimization after the main pretraining schedule. Our model family combines RoPE, RMSNorm, SwiGLU, tied embeddings, and optional grouped-query attention (GQA). The final Model 2 has 99.03M parameters, 16 layers, and a 1:2 GQA ratio. It is trained on a 50/20/15/15 mixture of FineWeb-Edu, Wikipedia, science, and books using a Muon-AdamW optimizer split. Its first training stage reaches 21.163 public-validation perplexity after 38k updates. Low-learning-rate continuation and dense checkpoint evaluation reduce this to **18.768** at step 69.7k, which we select for the final submission. Controlled comparisons show that data composition, optimizer choice, and checkpoint selection are as consequential as architecture at this scale.

1 Introduction

Problem definition. The competition asks for a causal language model that predicts each GPT-2 BPE token from its preceding context. The submitted model must contain at most 100M parameters, return logits over exactly 50,257 vocabulary entries, and finish evaluation within the course time limit. Performance is measured by token-level perplexity on a public validation set during development and a hidden test set for the final ranking.

Problem significance. Compact language models are useful when memory, latency, privacy, or deployment cost rules out a large hosted model. The same next-token objective supports document completion, educational tools, coding assistance, retrieval reranking, and on-device generation. The constrained setting is also scientifically useful: because model size is fixed, improvements must come from better capacity allocation, data, optimization, and engineering rather than parameter scaling alone.

Technical challenges. The parameter cap creates direct competition among vocabulary embeddings, depth, attention projections, and feed-forward width. Data presents a second challenge: a single source may train efficiently yet generalize poorly to the multi-domain evaluator. Long runs must also resume exactly, preserve optimizer and loader state, and export an inference-compatible model. Finally, validation performance near convergence is non-monotonic, so the last checkpoint is not necessarily the best checkpoint.

Contributions. Our work makes four concrete contributions:

- We compare three sub-100M decoder architectures that trade independent key/value capacity and residual width for depth.

- We implement a deterministic four-source loader and show that balanced multi-domain training outperforms single-source and overly FineWeb-heavy alternatives.
- We use Muon for hidden matrices and AdamW for embeddings and remaining parameters, improving early validation PPL by 8.6% over AdamW alone.
- We reduce Model 2 PPL from 21.163 to 18.768 without changing its architecture, data mixture, or objective, demonstrating the value of low-rate continuation and checkpoint selection.

2 Related Work

Compact Transformer design. Our models follow the decoder-only Transformer family [10] and the minimal training interface popularized by nanoGPT [4]. Modern components include rotary position embeddings [9], RMSNorm [11], SwiGLU [8], and GQA [1]. MobileLLM argues that deep-and-thin networks, embedding sharing, and GQA are particularly effective below one billion parameters [6]. We test the same capacity-allocation principle at a smaller 100M-parameter scale.

Training data and compute. FineWeb-Edu provides filtered educational web text and is a strong default for small-model pretraining [7]. However, data-mixture work emphasizes that domain weights materially affect performance under a fixed token budget [5]. Compute-optimal scaling results also show that model size cannot be considered independently of the number and quality of training tokens [2]. These findings motivate our controlled source and mixture comparisons rather than treating data as a fixed preprocessing choice.

Efficient optimization. The modded-nanoGPT community demonstrates that careful kernels, mixed precision, and alternative optimizers can substantially improve small-model training [3]. Our contribution is not a new optimizer; instead, we evaluate a practical split in which Muon handles two-dimensional hidden matrices while AdamW handles embeddings and other parameters, followed by conservative continuation schedules.

3 Method

3.1 Task and Data

For a token sequence $x_{1:T}$, the model estimates the conditional distribution $p_\theta(x_t \mid x_{<t})$. Training minimizes the corresponding next-token negative log-likelihood:

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log p_\theta(x_t \mid x_{<t}), \quad \text{PPL} = \exp(\mathcal{L}). \quad (1)$$

Inputs are contiguous 1024-token contexts and outputs are unnormalized logits over the GPT-2 vocabulary. We do not normalize token IDs or targets.

All sources are tokenized with GPT-2 BPE and stored as uint16 shards. The training set combines FineWeb-Edu, English Wikipedia, OpenWebMath-derived science text, and English Gutenberg books. The loader implements Table 1 as an exact 20-example cycle: 10, 4, 3, and 3 windows from the respective sources. Source cursors advance sequentially without overlap, while cycle order is reshuffled after each pass. Each 1025-token window yields 1024 inputs and one-position-shifted targets. The public `val.bin` is used only for evaluation and is never included in training.

Table 1: Final four-source sampling weights.

Source	Share
FineWeb-Edu	50%
Wikipedia	20%
Science/math	15%
Books	15%

Our central data hypothesis is that FineWeb-Edu supplies high-quality general web text, while Wikipedia, science, and books broaden domain coverage. We use FineWeb-Edu-only training as the fallback and compare against OpenWebText-only training and several FineWeb-heavy mixtures as negative controls in Section 4.5.

3.2 Model Family

The starter baseline is an 8-layer, 8-head, 512-dimensional GPT with learned absolute positions, LayerNorm, GELU feed-forward blocks, standard multi-head attention, tied embeddings, and AdamW. Our candidates retain the causal decoder structure but use RoPE, RMSNorm, SwiGLU, bias-free projections, zero dropout, and tied input/output embeddings. Figure 1 shows the three complete capacity allocations evaluated under the parameter cap.

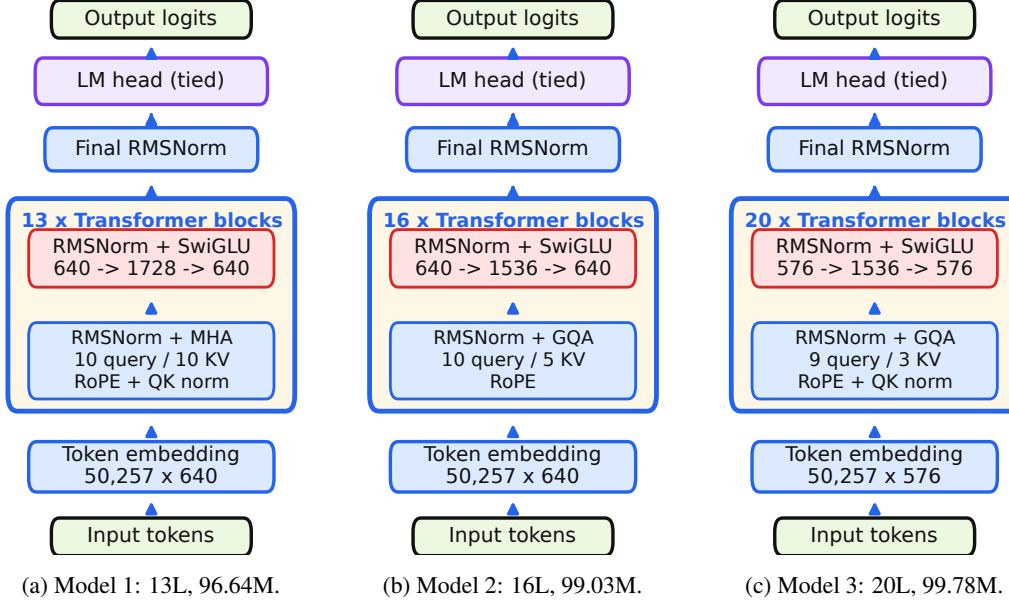


Figure 1: Three sub-100M capacity allocations. GQA lets Models 2 and 3 trade independent key/value projections for greater depth.

Table 2: Exact capacity allocations for the starter and three candidate architectures. Parameters count tied embeddings once.

Model	Layers	Width	Q/KV heads	MLP	QK norm	Parameters
Starter	8	512	8/8	2,048	No	< 100M
Model 1	13	640	10/10	1,728	Yes	96.64M
Model 2	16	640	10/5	1,536	No	99.03M
Model 3	20	576	9/3	1,536	Yes	99.78M

Block computation. For hidden state h_ℓ , each pre-normalized residual block computes

$$u_\ell = h_\ell + \text{Attn}(\text{RMSNorm}(h_\ell)), \quad (2)$$

$$h_{\ell+1} = u_\ell + W_{\text{down}}[\text{SiLU}(W_{\text{gate}}z_\ell) \odot W_{\text{up}}z_\ell], \quad z_\ell = \text{RMSNorm}(u_\ell). \quad (3)$$

RoPE is applied to queries and keys before causal scaled dot-product attention. The output projection and SwiGLU down projection return to the residual width, and the final language-model head shares weights with the token embedding. The implementation uses no projection biases or dropout.

Capacity rationale. Let d be the residual width and $r = h_{KV}/h_Q$. Ignoring biases, the four attention projections require approximately $d^2(2 + 2r)$ parameters per layer: d^2 each for queries and outputs and rd^2 each for keys and values. Standard MHA has $r = 1$ and costs $4d^2$; the 1:2 and 1:3 GQA layouts cost approximately $3d^2$ and $\frac{8}{3}d^2$. Models 2 and 3 reinvest those savings in three and seven additional layers relative to Model 1. Because width, MLP size, GQA ratio, and QK normalization also co-vary, the comparison supports complete deep-and-thin designs rather than attributing the result to GQA alone.

3.3 Optimization and Continuation

Muon updates the two-dimensional hidden-layer matrices, while AdamW handles embeddings and all other parameters. The main stage uses warmup followed by cosine decay over 19.92B sampled tokens. An AdamW-only pilot improved substantially more slowly.

Continuation restores weights, optimizer moments, data-loader cursors, and random-number states. For Model 2, the Muon rate decreases from roughly 3.0×10^{-4} at step 38k to 3.0×10^{-5} near step 69.7k; AdamW remains at 4% of that rate. Branches from 62k, 63.6k, and 69.6k shorten the remaining cosine horizon and evaluate more frequently near convergence. Architecture, batch, mixture, and objective remain fixed throughout; the resulting gains therefore isolate continuation and checkpoint selection from other modeling choices.

Table 3: Model 2 training configuration.

Hyperparameter	Value
Context length	1,024
Micro-batch	16 sequences
Global batch	524,288 tokens
Precision	bf16 autocast
Gradient clipping	1.0
Main-stage updates	38,000
Warmup	200 updates
Peak Muon LR	1.3×10^{-2}
Peak AdamW LR	5.2×10^{-4}
Main LR floor	10% of peak

Table 4: Model 2 schedule and checkpoint-search stages. Learning rates are effective Muon rates observed in the logs; AdamW uses 4% of each value. Monitoring gives the spacing between quick or official evaluations.

Stage	Step range	Muon LR	Monitoring	Representative PPL
Main training	0–38.0k	$1.3 \times 10^{-2} \rightarrow 1.3 \times 10^{-3}$	2,000	21.163
Broad continuation	38.0–62.0k	$3.00 \times 10^{-4} \rightarrow 1.35 \times 10^{-4}$	500–2,000	19.210
Short branch	62.0–63.6k	$\approx 7.7 \times 10^{-5}$	100	19.096
Late branch	63.6–69.6k	$5.10 \times 10^{-5} \rightarrow 5.00 \times 10^{-5}$	100	18.917 at 69.2k
Final search	69.6–71.8k	$\approx 3.0 \times 10^{-5}$	50	18.768 at 69.7k

Continuation as local search. The broad continuation determines whether the main checkpoint still has optimization headroom. Once gains slow, each short branch starts from a strong saved state, lowers the schedule scale, and increases measurement frequency. This procedure is not weight averaging or fine-tuning on a new dataset: every branch preserves the same model, source mixture, objective, optimizer partition, and loader position. The only deliberate changes are the remaining cosine horizon and the frequency of checkpoint evaluation.

3.4 Design Rationale and Risk Controls

Our hypothesis is that four changes compound: deeper GQA allocation, balanced domains, the Muon–AdamW split, and low-rate continuation. The main risks are interaction effects, validation overfitting, and unstable resume. Matched pilots, full-state checkpoints, official validation, and conservative rates control these risks. The starter model, FineWeb-Edu-only data, AdamW, and the 38k checkpoint remain usable fallbacks if a component fails.

4 Experiments

4.1 Baselines and Evaluation

We use four complementary baseline families: **(i) Starter GPT**, the 8-layer nanoGPT-style model with learned positions, LayerNorm, GELU, and AdamW; **(ii) single-source data**, FineWeb-Edu-only and OpenWebText-only pilots under a shared short-run budget; **(iii) the architecture family**, Models 1–3 under the same 50/20/15/15 mixture and 38k-update budget; and **(iv) an optimizer control**, AdamW versus Muon + AdamW for Model 2 under a matched early-training budget.

The course evaluator computes token-weighted cross entropy over contiguous 1024-token chunks in `val.bin` and exponentiates the mean. Every full evaluation covers 5,169,152 target tokens. Public-validation PPL is the primary development metric; the hidden test is reserved for the course ranking

and is not used for schedule decisions. Internal mixed-shard validation losses are used for trend monitoring but are not treated as directly comparable to official PPL.

4.2 Implementation Details

Training uses PyTorch on a single NVIDIA L40S GPU with 48GB memory, bf16 mixed precision, and scaled dot-product causal attention. Pure cross-entropy updates take approximately eight seconds, so the selected 69.7k lineage alone represents roughly 155 single-GPU hours before evaluation overhead. Additional candidate and branch runs increase the full project cost. Because the corpus is streamed and much larger than the sampled budget, an “epoch” is not a literal pass over all documents. We therefore report updates and sampled tokens; the best Model 2 checkpoint represents 36.54B sampled tokens.

Hyperparameter exploration includes depths {13, 16, 20}, FineWeb shares {50, 53, 56}%, standard MHA versus GQA, AdamW versus Muon + AdamW, and progressively smaller continuation rates. Baseline runs are not all matched in length; exact comparison budgets are listed in Appendix A. The training and exported configurations both use exactly 50,257 vocabulary entries, and the tied output head therefore returns the evaluator-required logit dimension directly.

4.3 Reproducibility and State Integrity

Deterministic sampling. The source loader is seeded with 2025 and stores the shuffled 20-window cycle, the position within that cycle, per-source shard indices and token cursors, and the random-generator state. Within each source, cursors advance by one non-overlapping context per process; reaching the end rotates to the next memory-mapped shard. On resume, the code verifies that the distributed world size matches the saved value before restoring the cursor state. These checks prevent a nominal continuation from silently replaying windows or changing the mixture.

Checkpoint and evaluation contracts. Training checkpoints contain model weights, both optimizer states, loader state, CPU and CUDA random-number states, architecture configuration, and the global step. Submission exports are separate minimal bundles containing `model.py`, `config.json`, and raw model weights. At selected steps, the training script exports this bundle and invokes the unmodified course evaluator, which writes PPL, mean loss, token count, parameter count, and evaluation time to JSON. Thus the plotted Model 2 points are linked to evaluator-compatible artifacts rather than reconstructed from noisy mini-batch losses.

4.4 Main Results

Table 5: Public-validation checkpoints in the final model-selection progression. Lower PPL is better.

Model	Stage	Data mix	Split	Step	PPL
Pilot baseline	short run	50/20/15/15	public val	5k	38.828
Model 1	main	56/18/13/13	public val	38k	21.609
Model 1	continuation	56/18/13/13	public val	50.75k	19.090
Model 2	main	50/20/15/15	public val	38k	21.163
Model 2	continuation	50/20/15/15	public val	69.7k	18.768

Table 5 summarizes the complete progression. The short-run mixed-data baseline remains at 38.828 PPL, whereas the matched 38k architecture runs reach the low 20s. Continuation improves Model 2 by another 2.395 PPL. We select the step-69.7k checkpoint with 18.768 PPL as the final submission.

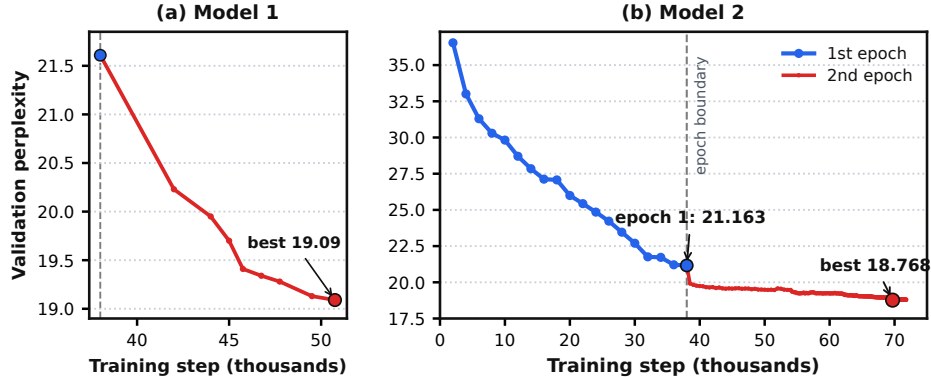


Figure 2: Low-rate continuation for Models 1 and 2. Red trajectories begin at the blue endpoints, preserving each checkpoint lineage. Model 1 values are team-reported; Model 2 values are log-verified.

Figure 2 shows that continuation transfers across architectures. Model 1 improves from 21.609 to 19.09 by step 50.75k, an 11.7% relative reduction. Model 2 reaches 19.666 at 42k, 19.210 at 62k, 19.012 at 66k, and 18.768 at 69.7k. Later checkpoints fluctuate near 18.79–18.81; Model 1 is similarly non-monotonic, worsening from 19.0900 to 19.1499 after another 500 updates. Evaluator-compatible checkpoint selection is therefore part of the method rather than a reporting convenience.

Table 6: Marginal returns during Model 2 continuation. Added tokens count sampled training tokens within each interval; gains are not independent because later intervals inherit earlier checkpoints.

Interval	Added tokens	Start PPL	End PPL	PPL reduction / B tokens
38k → 42k	2.10B	21.163	19.666	0.714
42k → 62k	10.49B	19.666	19.210	0.043
62k → 69.7k	4.04B	19.210	18.768	0.110

Training dynamics. The first 4k continuation updates deliver the largest return, reducing PPL by 1.497 with 2.10B additional tokens. The next 20k updates improve more slowly, whereas the short late branches recover a larger gain per token by lowering the rate and searching densely near a strong state. Branch selection itself consumes evaluation effort, so this is not proof of compute optimality; it does show that one checkpoint interval is poorly matched to both phases.

Checkpoint sensitivity. The best point is narrow rather than a stable terminal plateau. Official evaluation gives 18.768 at 69.7k, 18.776 at 70.0k, and 18.799 at 71.8k. Selecting the last state would discard a measurable verified gain without changing training compute, architecture, or data.

4.5 Controlled Comparisons

Table 7: Observed effect sizes within each controlled comparison or checkpoint lineage. Relative reduction uses the worse setting as the denominator; rows have different training budgets and are not additive.

Intervention	Comparison	Absolute PPL reduction	Relative reduction
Data source	OpenWebText → mixture	26.382	40.5%
Source weights	56/18/13/13 → 50/20/15/15	0.284	1.3%
Capacity allocation	Model 1 → Model 3	0.407	1.9%
Optimizer	AdamW → Muon + AdamW	3.433	8.6%
Model 1 continuation	38k → 50.75k	2.519	11.7%
Model 2 continuation	38k → 69.7k	2.395	11.3%

The effect-size view clarifies the project priorities. Choosing an unsuitable source overwhelms the architecture-scale differences, while the optimizer split and continuation each produce substantially larger relative gains than the matched 38k architecture change. Architecture still determines the state from which optimization continues, and a 0.4-PPL difference remains meaningful near the final operating point.

Table 8: Detailed validation ablations. Rows are matched within each factor; exact budgets appear in Appendix A. Lower PPL is better.

Factor	Setting	PPL
Data source	FineWeb-Edu only	39.670
	OpenWebText only	65.210
	Four-source mixture	38.828
FineWeb share	50/20/15/15	21.325
	53/19/14/14	21.460
	56/18/13/13	21.609
Architecture	Model 1: 13L, MHA	21.325
	Model 2: 16L, GQA 1:2	21.163
	Model 3: 20L, GQA 1:3	20.918
Optimizer	AdamW	39.973
	Muon + AdamW	36.540
LR branch	1.35×10^{-4}	19.218
	1.03×10^{-4}	19.152
	7.74×10^{-5}	19.103

Table 8 shows that the four-source mixture improves over FineWeb-Edu-only by 0.842 PPL and over OpenWebText-only by 26.382. Raising the FineWeb-Edu share from 50% to 53% or 56% hurts Model 1 by 0.135 and 0.284 PPL. Thus, the strongest source need not deserve a larger share.

At 38k, Model 2 beats Model 1 by 0.162 PPL, and Model 3 gains another 0.245. Because width and QK normalization also vary, this supports the complete deeper designs rather than GQA alone. In the matched optimizer pilot, Muon + AdamW reduces PPL from 39.973 to 36.540, an 8.6% relative gain.

Learning-rate refinement. We branch Model 2 three ways from the same 62k checkpoint and train each for 800 updates. Lowering the effective starting Muon LR from 1.35×10^{-4} to 1.03×10^{-4} and 7.74×10^{-5} improves PPL at 62.8k from 19.218 to 19.152 and 19.103, respectively. We retain the lowest-rate branch, which reaches 19.096 at 63.6k. Restarting at 5.10×10^{-5} then reaches 18.781 at 69.6k, and a final 3.01×10^{-5} branch reaches 18.768 after 100 updates. These later stages inherit selected checkpoints and use unequal budgets, so they document the successful search trajectory rather than independent matched ablations.

4.6 Negative Findings

Several negative results changed our procedure. OpenWebText-only training was not competitive; increasing the strongest source from 50% to 56% degraded Model 1; and the final continuation checkpoint was not the best checkpoint. Model 3 was strongest at 38k, beating Model 2 by 0.245 PPL, so we do not claim that Model 2 is the globally best architecture. We continued Model 2 because its complete model, optimizer, loader, and random states were available for a controlled resume. A fair final architecture ranking would require the same continuation and checkpoint-selection budget for all three models.

5 Discussion

Achievements and lessons. We train a competitive sub-100M model, reducing public-validation PPL from 38.828 in a short pilot to 18.768 at the selected submission checkpoint. Our architecture

study favors deeper GQA designs under a fixed parameter cap, while continuation yields the largest within-model improvement. More broadly, the model, data loader, optimizer, and checkpoint policy form one system; architecture-only optimization leaves substantial performance unused.

Selection validity. Repeated use of one public validation set creates selection bias, especially when checkpoints are evaluated every 50–100 updates. We mitigate this concern by keeping `val.bin` out of training and changing only schedule parameters after the architecture and data mixture are fixed. This does not quantify the full uncertainty introduced by branch selection. A stronger protocol would reserve a second local validation mixture for schedule decisions and use the public evaluator only for sparse confirmation.

Compute allocation. The selected lineage processes 19.92B tokens before continuation and another 16.62B by the best checkpoint. The marginal-return analysis shows that these tokens are not equally valuable: the first 2.10B continuation tokens produce most of the immediate gain, whereas later progress depends on smaller rates and denser measurement. Near convergence, a full official evaluation can take several minutes, so evaluating every saved checkpoint becomes a meaningful fraction of wall-clock cost. Coarse evaluation is appropriate early; dense evaluation is justified only once successive training updates move PPL by hundredths.

Systems lesson. Exact continuation required more than loading model weights. Restoring both optimizers, loader cursors, source-cycle order, and random states preserved the meaning of a global training step and avoided data replay. Separating rich training checkpoints from minimal submission bundles also reduced the risk of exporting optimizer state or an incompatible configuration. These engineering choices do not directly change the loss function, but they make the late-stage gains auditable and reproducible.

Bottlenecks and limitations. Full-scale controlled experiments are expensive, so not every candidate receives the same continuation budget. The architecture comparison changes multiple dimensions at once and should not be interpreted as a factorial ablation. Validation-driven branching can also over-specialize to `val.bin`; without an independent local holdout, we cannot quantify this effect. Experiment metadata is distributed across logs, model cards, and presentation records, making provenance harder to audit than a unified experiment tracker.

Future work. Future experiments should allocate equal sampled-token budgets and checkpoint-selection effort to all three architectures. A second held-out mixture would reduce schedule overfitting, while automated evaluation and best-checkpoint retention would remove manual branching. More systematic mixture optimization could replace the coarse 50/53/56% sweep, and matched throughput measurements could separate parameter efficiency from realized training speed, memory use, and inference latency under deployment constraints.

6 Team Contributions and LLM Usage

- **Yirui Miao:** experiment planning; analysis of data and architecture choices; ablation interpretation; report and presentation synthesis.
- **Fengfei Yu:** Model 2 training and continuation; evaluation and export infrastructure; log consolidation; result verification.
- **Tianyue Zhang:** data-mixture experiments; result synthesis; report analysis and writing.
- **Zavier Zhao:** Model 1 development; baseline and mixture experiments; presentation and report writing.

LLM usage. Large language models were used as writing and code-review assistants for summarizing local experiment notes, drafting report prose, and checking organization. The team remains responsible for all reported experimental claims, and numerical claims in this report were taken from the repository logs, model cards, code, or the team’s presentation notes.

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [2] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- [3] Keller Jordan. modded-nanogpt. <https://github.com/KellerJordan/modded-nanogpt>, 2024.
- [4] Andrej Karpathy. nanogpt. <https://github.com/karpathy/nanoGPT>, 2022.
- [5] Yajiao Liu, Congliang Chen, Junchi Yang, and Ruoyu Sun. Rethinking data mixture for large language models: A comprehensive survey and new perspectives. *arXiv preprint arXiv:2505.21598*, 2025.
- [6] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yanyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. *arXiv preprint arXiv:2402.14905*, 2024.
- [7] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.
- [8] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [9] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [11] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, 2019.

A Evaluation Budgets

Table 9: Matched evaluation points for the comparisons in Table 8.

Factor	Compared settings	Evaluation update
Data source	FineWeb-Edu, OpenWebText, mixture	5,000
FineWeb share	50%, 53%, 56% in Model 1	38,000
Architecture	Models 1–3, 50/20/15/15 mix	38,000
Optimizer	AdamW, Muon + AdamW in Model 2	2,000
LR branch	Three Model 2 rates from the same 62k state	62,800

B Architecture and Reproducibility Details

Table 10: Model 2 code, checkpoints, and final evaluation records.

Artifact	Repository location or value
Main training	build-nanogpt/train_gpt2_v13.py
Continuation	build-nanogpt/train_gpt2_v16.py
Main checkpoint	build-nanogpt/log/v13/train_ckpt_38000.pt
Best exported checkpoint	build-nanogpt/log/v16/.../submission_..._69700
Best public validation	Step 69,700; PPL 18.7681466; loss 2.932161 nats
Code repository	https://github.com/tylerz/cse251b-nanogpt